Exercises: Distributions

In the following, let us use the NLSY97 Jobs data file (available on Moodle): `nlsy97_jobs.parquet`.

I. Loading the data

1. Let us load the necessary packages:

```
# external (third-party) packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

import math
from pathlib import Path
```

2. Let us define the path to the project directory (I name it `ROOT` this time) and the path to the directory in which our data file is located (I name it `DATA` this time):

```
# Your path will almost certainly differ from the one below.
ROOT = r'C:\Users\PC_DS_ECON_5\Desktop\data-analytics-python-wise2627'

# The relative path could also differ from the one below.
DATA = ROOT + '/local/data'

print(ROOT)
print(DATA)
```

3. Let us read in the `nlsy97_jobs.parquet` file using `pandas.read_parquet()`:

```
df = pd.read_parquet(DATA + '/nlsy97_jobs.parquet')
```

(Reminder: To read the file, one needs a Parquet engine, such as the one provided by the `pyarrow` package. The package can be installed in the usual way with `pip install pyarrow` after activating the virtual environment.)

II. Inspecting and subsetting

(a) What is the unit of observation in our dataset?

- `person_id` X `year`
- `person_id` X `job_rank`
- `person_id` X `year` X `job_rank`

```
# Your code goes here
```

(b) Let us keep observations from the most recent year using `.loc`, `eq()`. How would you use `drop_duplicates()` and `max()` to find out which year is the most recent?

```
df1 = (
    df
        # Your code goes here
        .reset_index(drop=True)
        .copy()
)
```

(c) Let us further restrict the dataset by keeping only the most salient job for each individual, i.e. jobs with `job_rank==1`. Use `.loc` and `eq()` to apply the filtering to dataframe `df1`.

```
df1 = (
    df1
        # Your code goes here
        .reset_index(drop=True)
        .copy()
)
```

(d) Verify, using `is_unique`, whether there is only one observation per individual in the remaining dataset `df1`.

```
# Your code goes here
```

(e) Use `value_counts()` to display the unique values of the variable `ethnicity` and their distribution in the sample.

```
# Your code goes here
```

(f) Create a variable named `log_hpay` such that it equals the logarithm of hourly pay (variable `hpay`) if hourly pay is not missing and positive, otherwise taking on a missing value (`pd.NA`). For conditional assignement, you can either use `.loc` statements or the `np.where()` function from `numpy`. For specifying the conditions, you may want to you use `gt()` and `notna()`. For taking logarithm, you can use `np.log()` from `numpy`.

```
# Your code goes here
```

Finally, let us make sure that `log_hpay` has a numerical data type (technically `Float64`).

```
df1['log_hpay'] = df1['log_hpay'].astype('Float64')
```

III. Relative frequencies of log hourly pay by ethnicity

In what follows, we would like to describe the distribution of log hourly pay by ethnicity.

Let us begin by plotting the distributions.

Distributions? What does that mean? We would like to plot relative frequencies (the finite-sample equivalents of probabilities) for different values of our variable of interest.

Log hourly pay is a *continuous* variable: in principle, log hourly pay can take on any real value, because hourly pay can take on any positive value.

Continuous variables are to be contrasted with discrete variables: a *discrete* variable takes on a finite or countably infinite number of values.

To visualise the distribution of a continuous variable (such as log hourly pay), it makes sense to group the values into intervals of equal width. This is called *binning*. We can then display the relative frequencies for these intervals in a *histogram*.

Creating a histogram for a continuous variable usually involves the following steps:

- Define the intervals (bins).
- Group observations according to the interval they fall into.
- Compute the relative frequency for each interval.
- Plot the relative frequencies for each interval.

Let us define the width of the intervals as `binwidth=0.5`, as we are dealing with values in logs here.

We can now define the interval bounds or cut points.

For this purpose, it makes sense to use `np.arange()`. Let us start the range of cutpoints a little below the minimum value and stop a little above the maximum, to make sure that every observed value of our variable belongs to an interval.

```
binwidth = 0.5
bins = np.arange(
    start=df1['log_hpay'].min() - binwidth,
    stop=df1['log_hpay'].max() + 2 * binwidth,
    step=binwidth
)
```

For definig the binned variable `log_hpay_bin`, let us use the function `pd.cut()`.

```
df1['log_hpay_bin'] = pd.cut(
    x=df1['log_hpay'],
    bins=bins
)
```

(a) What is the data type of the variable `log_hpay_bin`?

```
# Your code goes here
```

Let us turn to computing the relative frequencies by bin within each category of `ethnicity`.

(b) Use `pd.crosstab()` to obtain a table with the rows corresponding to the log hourly pay bins (`log_hpay_bin`), the columns corresponding to the categories of the variable `ethnicity`, and the relative frequencies being calculated within each column.

```
df2 = pd.crosstab(
    index = # Your code goes here
    columns = # Your code goes here
    normalize = # Your code goes here
).reset_index()
df2.columns.name = None
```

Before plotting, there is one more step that we need to take. On the horizontal axis of our histogram, we would like to display the bins. In `matplotlib`, this does not work with intervals being defined by both a lower bound and an upperbound. The most direct workaround is to create a variable `log_hpay_bin_mid` that corresponds to the midpoint of each log hourly pay interval. Single midpoint values can be easily used as axis values in matplotlib. Let us create the midpoint variable with `transform()` using the following code:

```
df2['log_hpay_bin_mid'] = (
    df2['log_hpay_bin'].transform(lambda interval: interval.mid).astype('Float64')
)
```

It is time for plotting. Let us plot the distribution for three categories of `ethnicity`: `Black`, `Hispanic`, `Non-Black / Non-Hispanic`. (Let us omit the fourth mixed category, because it is too small in this sample.)

There are at least two possibilities: A. Three subplots below each other. B. Only one plot for all categories, bars side-by-side within each bin.

Let us produce three subplots below each other using the following code:

```
fig, axs = plt.subplots(nrows=3, ncols=1, sharex='all', figsize=(4,6))
axs[0].bar(
    x=df2['log_hpay_bin_mid'],
    height=df2['Black'],
    width=0.5 * binwidth
)
axs[0].set_title('Black')
axs[1].bar(
    x=df2['log_hpay_bin_mid'],
    height=df2['Hispanic'],
    width=0.5 * binwidth
)
axs[1].set_title('Hispanic')
axs[2].bar(
    x=df2['log_hpay_bin_mid'],
    height=df2['Non-Black / Non-Hispanic'],
    width=0.5 * binwidth
```

```
)
axs[2].set_title('Non-Black / Non-Hispanic')
fig.supylabel('Relative Frequency')
fig.supxlabel('Log hourly pay')
fig.tight_layout()
```

(c) Rewrite the code above using a `for` loop to avoid some repetitions. You can for example try it the following way:

```
fig, axs = plt.subplots(nrows=3, ncols=1, sharex='all')
for i, ethnicity in enumerate(['Black', 'Hispanic', 'Non-Black / Non-Hispanic']):
    # your code goes here



fig.supylabel('Relative Frequency')
fig.supxlabel('Log hourly pay')
fig.tight_layout()
```

Let us also produce a plot with the bars next to each other in different colours:

```
fig, ax = plt.subplots()
ax.bar(
    x=df2['log_hpay_bin_mid'] - 0.2 * binwidth,
    height=df2['Black'],
    width=0.2 * binwidth,
    label='Black'
)
ax.bar(
    x=df2['log_hpay_bin_mid'],
    height=df2['Hispanic'],
    width=0.2 * binwidth,
    label='Hispanic'
)
ax.bar(
    x=df2['log_hpay_bin_mid'] + 0.2 * binwidth,
    height=df2['Non-Black / Non-Hispanic'],
    width=0.2 * binwidth,
    label='Non-Black / Non-Hispanic'
)
ax.set_ylabel('Relative frequency')
ax.set_xlabel('Log hourly pay')
ax.legend()
fig.tight_layout()
```

(d) Rewrite the code above using a for loop to avoid some repetition. For example, you can try it in the following way:

```
fig, ax = plt.subplots()
barshift = [(-0.2) * binwidth, 0, (0.2) * binwidth]
groups = ['Black', 'Hispanic', 'Non-Black / Non-Hispanic']
for i, group in enumerate(groups):
    # your code goes here



ax.set_ylabel('Relative frequency')
ax.set_xlabel('Log hourly pay')
ax.legend()
fig.tight_layout()
```

IV. Cumulative relative frequencies of log hourly pay by ethnicity

Comparing the three distributions can also be done in a more direct way using *cumulative relative frequencies*. Their advantage is that they are easily defined for both continuous and discrete variables.

The cumulative relative frequency of a value y of log hourly pay is simply the share/proportion of observations with a log hourly pay less than or equal to y. The population equivalent of cumulative relative frequencies are cumulative probabilities, yielding a simple formal statement:

$$\text{Cumulative probability of } log_hpay \text{ associated with the value } y \ = \mathbb{P}[log_hpay \leq y]$$

In particular, we would be interested in cumulative probabilities / relative frequencies conditional on ethnicity:

$$\text{Cumulative probability of } log_hpay \text{ associated with the value } y \text{ conditional on ethnicity} = \mathbb{P}[log_hpay \leq y | ethnicity]$$

(a) In a first step towards computing cumulative relative frequencies, let us group observations by `ethnicity` using `groupby()` and compute the relative frequency associated with each unique value of `log_hpay` using `value_counts()`. Complete the code below.

```
df3 = (
    df1
        # your code goes here
        .reset_index()
        .copy()
)
```

```
df3 = (
    df1
        .groupby(by='ethnicity')
        .value_counts(
            subset=['log_hpay'],
            normalize=True
        )
        .reset_index()
```

```
        .copy()
)
```

In a second step, let us sort our dataset in ascending order by the grouping variable `ethniticity` and the variable of interest `log_hpay` in ascending order using `sort_values()`:

```
df3 = (
    df3
        .sort_values(
            by=['ethnicity', 'log_hpay'],
            ascending=True,
            ignore_index=True
        )
        .copy()
)
```

(b) Once sorted, let us group observations by `ethnicity` while selecting the column `proportion` (containing the relative frequencies) and compute cumulative sums on it using `cumsum()` to obtain the cumulative relative frequencies. Let us assign the result to a new variable `cumulative_proportion`. Complete the code below:

```
df3['cumulative_proportion'] = (
    df3
        .groupby(by=['ethnicity'])['proportion']
        # your code goes here
        .copy()
)
```

It is time for plotting. Let us plot the cumulative relative frequencies by ethnicity on a single graph. Put differently, let us plot the empirical cumulative distribution function of log hourly pay by ethnicity.

One thing to keep in mind: For each plot, we need to make sure that we use only the relevant part of the dataset for the given plot. To split our dataset, let us create a dictionary:

```
ethnicities = df3['ethnicity'].drop_duplicates()
df3s = {}
for ethnicity in ethnicities:
    df3s[ethnicity] = (
        df3
            .loc[df3['ethnicity'].eq(ethnicity),:]
            .reset_index(drop=True)
            .copy()
    )
```

All is now ready for proceeding with plotting. An empirical cumulative distribution function is a step function. A technical detail: because of the weak inequality $\leq$ ("less than of equal to") in the conventional definition, the steps should continue until the next value. This is why `where='post'` is specified in the code below.

```
fig, ax = plt.subplots()
for ethnicity in ['Black', 'Hispanic', 'Non-Black / Non-Hispanic']:
    ax.step(
        x=df3s[ethnicity]['log_hpay'],
        y=df3s[ethnicity]['cumulative_proportion'],
        where='post',
        label=ethnicity
    )
ax.legend()
ax.set_xlabel('Log hourly pay')
ax.set_ylabel('Cumulative relative frequency')
ax.set_title('Cumulative distribution functions')
```

It might be the case that we do not like hourly pay being expressed in log points. We could for example preserve the log-scale while displaying hourly pay in dollars and not log-dollars. One option is to choose a set of specific values of hourly pay for which we wish to have ticks and then translate them into log hourly pay for the placement of the ticks:

```
fig, ax = plt.subplots()
for ethnicity in ['Black', 'Hispanic', 'Non-Black / Non-Hispanic']:
    ax.step(
        x=df3s[ethnicity]['log_hpay'],
        y=df3s[ethnicity]['cumulative_proportion'],
        where='post',
        label=ethnicity
    )
ax.legend()
ax.set_xlabel('Hourly pay (log scale)')
ax.set_ylabel('Cumulative relative frequency')
ax.set_title('Cumulative distribution functions')
# setting ticks
tick_labels = [1,10,30,100]
tick_values = np.log(tick_labels)
ax.set_xticks(
    ticks=tick_values,
    labels=tick_labels
)
```

This graph shows the typical pattern found: statistically speaking, when employed, black individuals tend to be paid less than hispanics, who statistically speaking tend to be paid less than non-black/non-hispanic (mostly white) individuals. This is a fully descriptive and unconditional analysis. The possible reasons for these differences remain unexplained by the plot itself. One can nevertheless think of different factors contributing to these broad patterns.

V. Quantiles of log hourly pay by ethnicity

In the context of describing distributions of log hourly pay by ethnicity, we could flip the perspective when considering cumulative relative frequencies / probabilities.

Instead of asking, what is the share of individuals p with log hourly pay lower than or equal to y, we could ask what is the value of log hourly pay y such that approximately p-proportion of individuals have a log hourly pay lower than or equal to y.

In the first (former) case, we ask what is the cumulative probability / proportion / share / relative frequency associated with a certain value y. This was the subject of exercise IV.

In the second (latter) case, we ask what is the value such that the cumulative probability / proportion / share / relative frequency associated with it is p. This value is called the *p-quantile* and is the subject of the present exercise.

Formally, by convention, using the equivalence between relative frequencies and probabilities, the following definition ensures that the quantiles are defined for any value of p:

p-quantile of log hourly pay `log_hpay` conditional on ethnicity: minimum value of y such that $\mathbb{P}[log_hpay \leq y|ethnicity] \geq p$

There are, however, other conventions as well. In most cases of interest, the differences across the methods should be marginal.

Conceptually, the quantile function is the inverse of the cumulative distribution function. So plotting the quantile function boils down to starting from the plot of the cumulative distribution function (the plot from the end of exercise IV) and swapping the horizontal and vertical axes.

(a) Let us create a plot of the quantile function of log hourly pay by ethnicity by implementing the swapping of the axes. Complete the code below to ensure that the correct quantities are displayed on the horizontal and vertical axes.

```
fig, ax = plt.subplots()
for ethnicity in ['Black', 'Hispanic', 'Non-Black / Non-Hispanic']:
    ax.step(
        # your code goes here

        where='pre',
        label=ethnicity
    )
ax.legend()
ax.set_ylabel('p-quantile of hourly pay (log scale)')
ax.set_xlabel('Cumulative relative frequency (p)')
# setting ticks
tick_labels = [1,10,30,100]
tick_values = np.log(tick_labels)
ax.set_yticks(
    ticks=tick_values,
    labels=tick_labels
)
ax.set_title('Quantile functions')
```

There are many well-known quantiles.

- *median*: the 0.5-quantile
- *lower quartile*: the 0.25-quantile
- *upper quartile*: the 0.75-quantile

For describing distributions of income, typical quantiles are quintiles:

- *quintile*s: 0.2-quantile, 0.4-quantile, 0.6-quantile, 0.8-quantile
- *decile*s: 0.1-quantile, 0.2-quantile, 0.3-quantile, 0.4-quantile, 0.5-quantile, 0.6-quantile, 0.7-quantile, 0.8-quantile, 0.9-quantile

And, of course, *percentile*s are also quantiles.

`pandas` has a dedicated function `quantile()` for calculating specific quantiles using the original dataset.

(b) Let us compute the median of hourly pay by ethnicity as follows: - let us first group observations by `ethnicity` using `groupby()` while selecting the `log_hpay` column; - let us then apply the `quantile()` function with the relevant value; - let us then 'save' the resulting series as `medians`; - let us then display the quantiles to dollars using the exponential function `np.exp()` from `numpy`.

Complete the code below.

```
medians = (
    df1
        # your code goes here
)
np.exp(medians)
```

Finally, let us display the medians on our quantile functions plot.

```
fig, ax = plt.subplots()
hpay_lower_bound = 5
ethnicities = ['Black', 'Hispanic', 'Non-Black / Non-Hispanic']
for ethnicity in ethnicities:
    ax.step(
        x=df3s[ethnicity]['cumulative_proportion'],
        y=df3s[ethnicity]['log_hpay'],
        where='pre',
        label=ethnicity
    )
# horizontal lines
for ethnicity in ethnicities:
    ax.hlines(
        y=medians[ethnicity],
        xmin=0,
        xmax=0.5,
        color='red',
        linestyle='--')
# vertical line
ax.vlines(
    x=0.5,
    ymin=np.log(hpay_lower_bound),
    ymax=medians['Non-Black / Non-Hispanic'],
    color='red',
    linestyle='--'
)
```

```
ax.legend()
ax.set_ylabel('p-quantile of hourly pay (log scale)')
ax.set_xlabel('Cumulative relative frequency (p)')

# setting axis limits
ax.set_ylim(bottom=np.log(hpay_lower_bound))
ax.set_xlim(left=0, right=1)

# setting ticks
tick_labels = [10,20,25,30,100]
tick_values = np.log(tick_labels)
ax.set_yticks(
    ticks=tick_values,
    labels=tick_labels
)

ax.set_title('Quantile functions')
```